ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP NHÓM TUẦN 3

HỆ ĐIỀU HÀNH (CO2017)

ĐỀ TÀI: THỰC HÀNH ĐA LUỒNG TỐT QUA BÀI TOÁN ƯỚC LƯỢNG π BẰNG MONTE CARLO

Lớp: [TODO: Lxx] – Nhóm: [TODO: xx]

GVHD: [TODO: tên giảng viên]

MSSV	Họ và tên	Việc
[TODO: MSSV]	**[TODO: Họ tên]** (longquawns)	1
[TODO: MSSV]	**[TODO: Họ tên]** (AnhTu97ccmk)	2
[TODO: MSSV]	**[TODO: Họ tên]** (Hungluu1702)	3
[TODO: MSSV]	**[TODO: Họ tên]** (ggg-maker-pixel)	4
2452081	Phạm Tuấn Anh (EigenKai)	5

Thành phố Hồ Chí Minh, tháng 11 năm 2026

BẢNG PHÂN CÔNG NHIỆM VỤ

Nhóm làm việc trên một repository chung, mỗi thay đổi đi qua một pull request và được review trước khi merge: `https://github.com/PTAxHVA/os-week3-monte-carlo-pi`.

Thành viên	Công việc	Tỉ lệ
[TODO: Họ tên] (Việc 1)	`common.h` (giao ước chung), Approach 1, `Makefile`, README, ghép và nộp bài	100%
[TODO: Họ tên] (Việc 2)	Approach 2 (biến đếm cục bộ, map–reduce); phân tích A, B, C, thí nghiệm 6.2	100%
[TODO: Họ tên] (Việc 3)	Approach 3 (mutex, tham số k); phân tích D, E, thí nghiệm 6.1, 6.3	100%
[TODO: Họ tên] (Việc 4)	Chạy toàn bộ phép đo trên máy đo chung, xuất số liệu và đồ thị	100%
Phạm Tuấn Anh (Việc 5)	Review mọi pull request, script đo và vẽ, thí nghiệm độ chính xác, viết báo cáo	100%

MỤC LỤC

BẢNG PHÂN CÔNG NHIỆM VỤ **1**

1. Mục tiêu **5**

2. Phương pháp **5**

2.1 Ước lượng π bằng Monte Carlo 5

2.2 Môi trường đo 5

2.3 Quy trình đo 6

3. Thiết kế ba approach **7**

3.1 Approach 1 – một luồng 7

3.2 Approach 2 – nhiều luồng, đếm cục bộ (map–reduce) 7

3.3 Approach 3 – biến đếm chung có mutex 8

3.4 Kiểm tra đúng sai 9

4. Kết quả **9**

4.1 Thời gian, speedup và hiệu suất 9

4.2 Độ chính xác của π 11

5. Phân tích **13**

5.1 A. Speedup không tuyến tính 13

5.2 B. Điểm bão hoà 14

5.3 C. Quá tải luồng (oversubscription) 14

5.4 D. Chi phí đồng bộ 14

5.5 E. Hiệu suất song song 15

6. Thí nghiệm đánh đổi **15**

6.1 Tần suất đồng bộ 15

6.2 Mở rộng số luồng 17

6.3 Tính cục bộ và tính dùng chung 17

7. Các thực hành tốt rút ra **17**

8. Kết luận **18**

Sử dụng công cụ AI **18**

DANH SÁCH HÌNH

1 Thời gian chạy theo số luồng (thang log, thanh sai số là độ lệch chuẩn) 10
2 Speedup theo số luồng, so với đường lý tưởng $S = N$ 10
3 Hiệu suất song song $E = S/N$ 11
4 Thời gian Approach 3 theo khoảng đồng bộ k ($N = 4$), đường ngang là Approach 2 16

DANH SÁCH BẢNG

1 Máy đo 6
2 Tham số đo 6
3 Thời gian (trung bình 5 lần), speedup và hiệu suất; cột mutex là $k = 1$ 9
4 π của 5 lần chạy với 5 seed khác nhau, $n = 10^7$, $N = 4$ (mutex dùng $k = 1000$) 12
5 `pi_big`, $N = 4$, seed 42 (`results/precision.csv`) 13
6 Approach 3 với $N = 4$ theo k; Approach 2 cùng N mất 0,0287 s . 16
7 So sánh Approach 2 và Approach 3 ($k = 1$) 17

1. Mục tiêu

Bài tập ước lượng π bằng phương pháp Monte Carlo theo ba cách: một luồng, nhiều luồng với biến đếm cục bộ, và nhiều luồng dùng chung một biến đếm có khoá mutex. Bài toán π chỉ là phương tiện. Thứ cần quan sát là:

- thực thi song song cải thiện hiệu năng tới đâu, và vì sao dừng lại;
- chi phí của đồng bộ hoá và giao tiếp giữa các luồng;
- cách thiết kế chương trình đa luồng đúng;
- hành vi đo được liên hệ thế nào với bộ định thời của hệ điều hành và giới hạn phần cứng.

2. Phương pháp

2.1 Ước lượng π bằng Monte Carlo

Lấy điểm (x, y) ngẫu nhiên đều trong hình vuông $[0,1]^2$. Điểm nằm trong phần tư hình tròn bán kính 1 khi $x^2 + y^2 \leq 1$. Phần tư hình tròn chiếm $\pi/4$ diện tích hình vuông, nên

$$\pi \approx 4 \cdot \frac{\text{số điểm trong hình tròn}}{\text{tổng số điểm}}. \tag{1}$$

Đề bài vẽ hình trên $[-1,1]^2$ với cả hình tròn; tỉ lệ diện tích vẫn là $\pi/4$ nên hai cách cho cùng một ước lượng. Với n điểm, sai số chuẩn của ước lượng là

$$\sigma_{\hat{\pi}} = 4\sqrt{\frac{p(1-p)}{n}} \approx \frac{1{,}642}{\sqrt{n}}, \qquad p = \frac{\pi}{4}, \tag{2}$$

khoảng $5{,}2 \cdot 10^{-4}$ khi $n = 10^7$.

2.2 Môi trường đo

Mọi phép đo thời gian chạy trên **một máy duy nhất** (yêu cầu của giảng viên), lấy từ `results/machine.txt`.

Bảng 1: *Máy đo*

Máy	Máy ảo VMware trên laptop AMD Ryzen 5 7235HS (4 nhân / 8 luồng)
CPU cấp cho máy ảo	4 vCPU (1 socket $\times$ 4 nhân, 1 luồng mỗi nhân), `nproc` = 4
Hệ điều hành	Ubuntu, nhân Linux 7.0.0-38-generic, x86_64
Trình biên dịch	gcc 15.2.0, cờ `-O2 -Wall -pthread`
Thời điểm đo	02/11/2026, 16:11

Lần đo đầu (15:58) dùng máy ảo 2 vCPU. Đường speedup khi đó chỉ có một bước tăng ($1 \rightarrow 2$ luồng), nên nhóm cấp lại 4 vCPU và đo lại toàn bộ. Báo cáo chỉ dùng bộ số 4 vCPU. Không cấp đủ 8 luồng cho máy ảo để máy chủ Windows còn CPU riêng, tránh nhiễu do máy ảo tranh CPU với chính máy chủ.

2.3 Quy trình đo

Bảng 2: *Tham số đo*

Số điểm mỗi lần chạy	$n = 10^7$, giống nhau cho mọi cấu hình
Seed	42; luồng thứ `tid` dùng seed `42 + tid`
Số luồng	1, 2, 4, 8, 16, 32, 64, 128
Số lần chạy	5 lần ghi nhận mỗi cấu hình; Approach 1 và 2 có thêm 1 lần chạy làm nóng bị bỏ
Giá trị báo cáo	trung bình và độ lệch chuẩn của 5 lần
Đồng hồ	`clock_gettime(CLOCK_MONOTONIC)`, thời gian thực
Khoảng đo	từ ngay trước `pthread_create` đầu tiên tới ngay sau `pthread_join` cuối và bước cộng gộp
Không đo	đọc tham số, khởi động tiến trình, in kết quả

Tạo và join luồng nằm trong khoảng đo vì đó là chi phí thật của việc chạy song song. Không dùng `clock()` vì hàm này cộng thời gian CPU của mọi luồng, tăng theo số luồng ngay cả khi thời gian thực giảm. Không dùng `rand()` vì glibc

bảo vệ trạng thái dùng chung của nó bằng một khoá nội bộ, sẽ thêm tranh chấp ẩn vào mọi approach. Toàn bộ quy trình nằm trong `scripts/run_bench.sh`; `scripts/plot.py` tính trung bình và vẽ đồ thị.

Chỉ số:

$$S(N) = \frac{T_{single}}{T_{parallel}(N)}, \qquad E(N) = \frac{S(N)}{N}, \tag{3}$$

với T_{single} là thời gian trung bình của Approach 1.

3. Thiết kế ba approach

Ba chương trình cùng tuân theo một giao ước (`src/common.h`): điểm lấy trong $[0,1]^2$ bằng `rand_r` với seed riêng; phần dư n mod N chia cho các luồng đầu; mỗi chương trình in đúng một dòng CSV `approach,threads,k,points,inside,pi,t`. Nhờ dùng cùng cách chia điểm và cùng seed, với cùng N, Approach 2 và 3 sinh ra *đúng cùng một tập điểm*, nên số điểm trúng (`inside`) phải bằng nhau. Đây là phép kiểm tra đúng sai chính.

3.1 Approach 1 – một luồng

Một vòng lặp sinh n điểm và đếm số điểm trúng. Không in gì trong vòng lặp. Thời gian trung bình của nó là T_{single}.

3.2 Approach 2 – nhiều luồng, đếm cục bộ (map–reduce)

Mỗi luồng nhận một struct riêng (số điểm, seed, ô kết quả), đếm vào một biến trên stack của chính nó và chỉ ghi kết quả ra struct một lần ở cuối. Luồng chính join rồi cộng các kết quả.

Vòng lặp của Approach 2 (src/pi_local.c)

```
static void *worker(void *arg_ptr) {
    thread_arg_t *arg = (thread_arg_t *)arg_ptr;
    unsigned int seed = arg->seed;          /* private seed:
    base + tid */
    long long local_inside = 0;              /* stack
   variable */
    for (long long i = 0; i < arg->points; i++) {
        double x = rand01(&seed), y = rand01(&seed);
```

```
        if (x * x + y * y <= 1.0) local_inside++;
    }
    arg->inside = local_inside;              /* written back
    once */
    return NULL;
}
```

Biến đếm nằm trên stack chứ không phải phần tử `count[tid]` của một mảng dùng chung. Các phần tử liền nhau của mảng nằm cùng một cache line; nếu mỗi luồng ghi vào đó ở mọi vòng lặp thì các nhân sẽ liên tục làm mất hiệu lực cache của nhau (*false sharing*).

3.3 Approach 3 – biến đếm chung có mutex

Biến toàn cục `g_inside` được bảo vệ bởi `pthread_mutex_t`. Mỗi luồng gom một lô nhỏ rồi cộng vào `g_inside` dưới khoá sau mỗi k vòng lặp, cộng thêm một lần cho phần dư. Với $k = 1$ (Approach 3 chuẩn của đề) luồng khoá mutex *ở mọi vòng lặp*; k lớn hơn là thí nghiệm đánh đổi ở mục 6.1.

Vòng lặp của Approach 3 (src/pi_mutex.c)

```
static int add_batch(long long batch) {
    int rc = pthread_mutex_lock(&g_lock);
    if (rc != 0) return rc;
    g_inside += batch;                       /* critical
   section */
    return pthread_mutex_unlock(&g_lock);
}
/* inside worker(): */
for (long long i = 0; i < args->n_points; i++) {
    double x = rand01(&seed), y = rand01(&seed);
    if (x * x + y * y <= 1.0) batch++;
    if ((i + 1) % args->k == 0) { args->error = add_batch(
   batch); batch = 0; }
}
```

3.4 Kiểm tra đúng sai

- Ở cả 8 số luồng, Approach 2 và Approach 3 (mọi k) cho cùng một giá trị `inside`; với $N = 1$ giá trị đó trùng Approach 1 (7 851 330). Kiểm trên toàn bộ 110 dòng của `results/raw.csv`.

- Khi bỏ khoá khỏi Approach 3 ($N = 4$, $k = 1$), ba lần chạy cho `inside` = 1 666 250; 1 707 553; 1 746 813 thay vì 7 853 800, tức $\hat{\pi} \approx 0{,}67$–$0{,}70$. Khoảng 78 % lượt cộng bị mất do *race condition*, và mỗi lần chạy mất một lượng khác nhau. (Bản thử nghiệm này không nằm trong repo; đo trên máy ảo Ubuntu 4 vCPU khác máy đo, chỉ dùng làm bằng chứng đúng sai.)

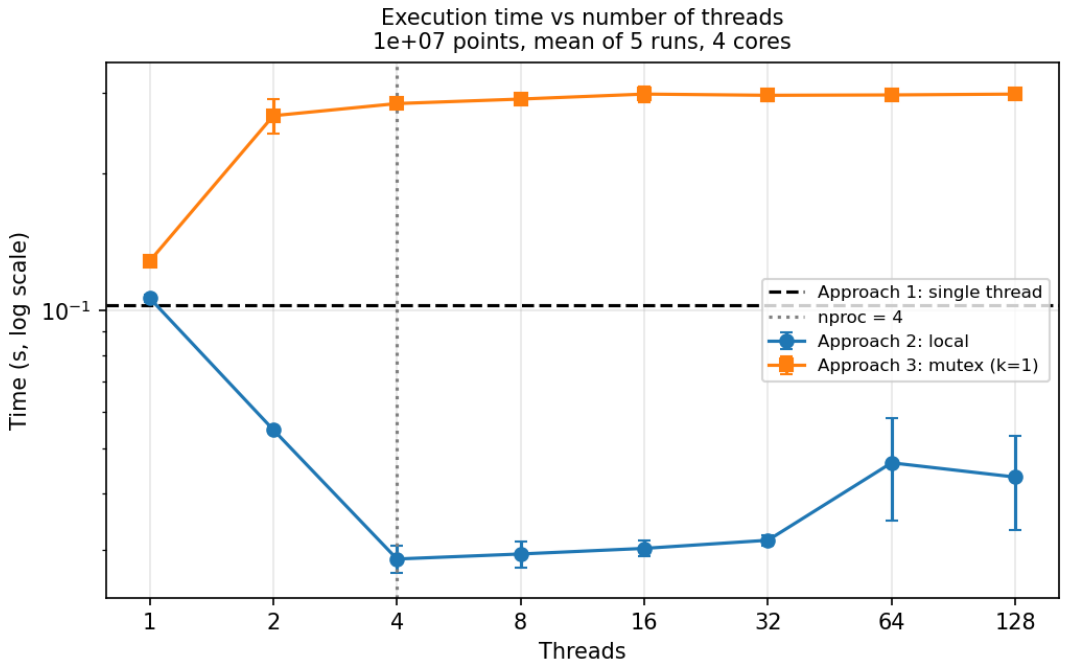
4. Kết quả

4.1 Thời gian, speedup và hiệu suất

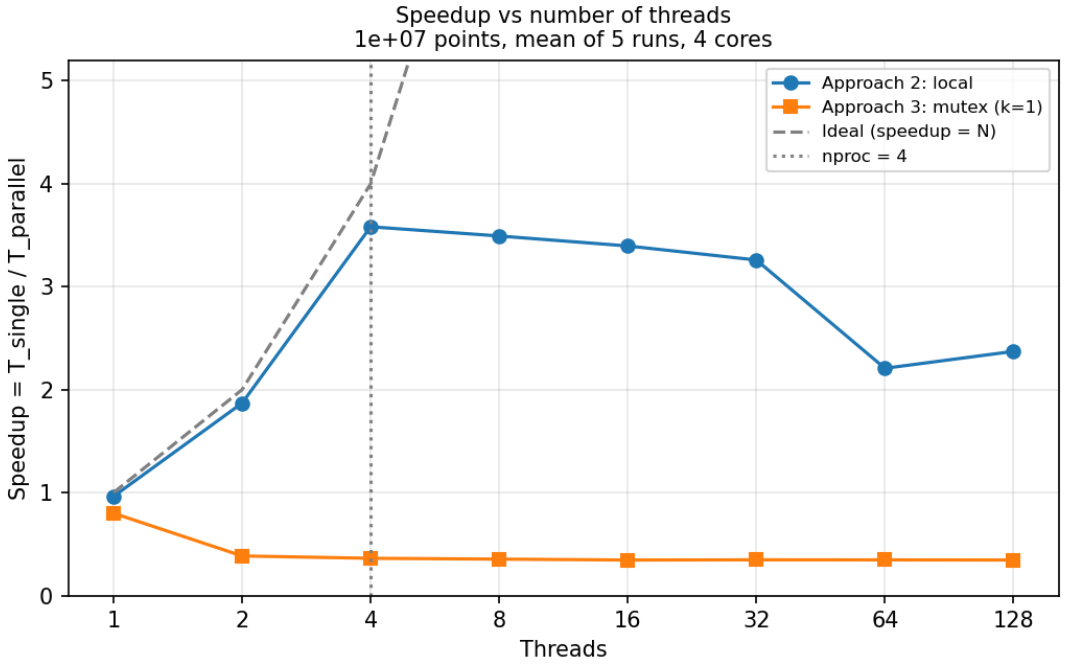
$T_{single} = 0{,}1028$ s (trung bình 5 lần, độ lệch chuẩn 0,0008 s).

Bảng 3: *Thời gian (trung bình 5 lần), speedup và hiệu suất; cột mutex là $k = 1$*

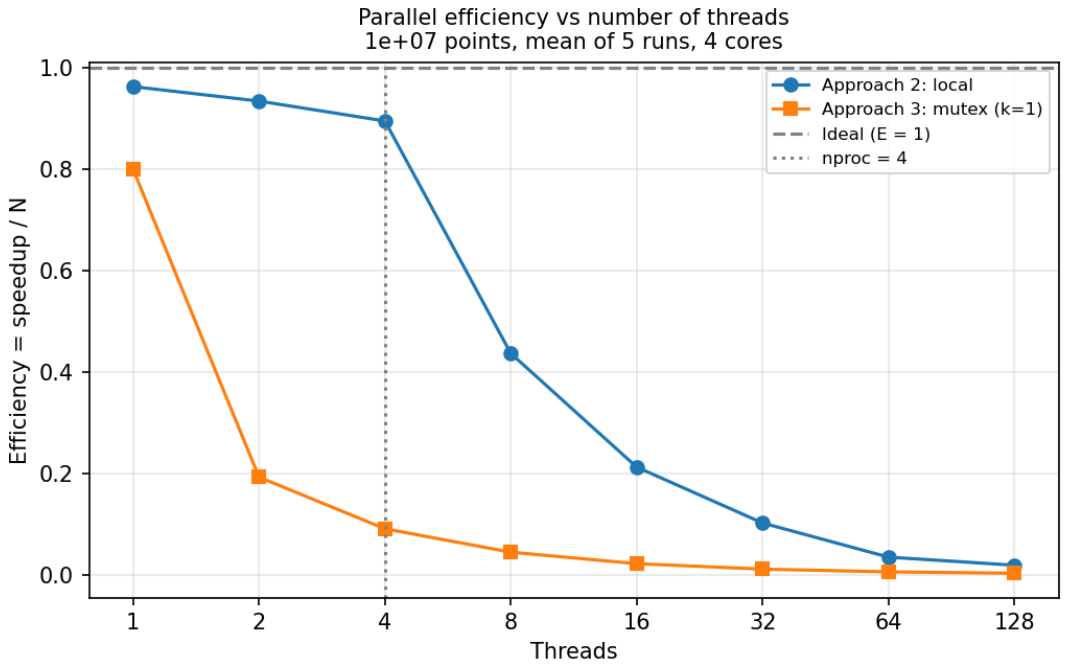
	Approach 2 – cục bộ			Approach 3 – mutex, $k = 1$		
N	T (s)	S	E	T (s)	S	E
1	0,1066	0,96	0,96	0,1285	0,80	0,80
2	0,0550	1,87	0,93	0,2670	0,38	0,19
4	**0,0287**	**3,58**	0,90	0,2838	0,36	0,09
8	0,0294	3,49	0,44	0,2903	0,35	0,044
16	0,0303	3,40	0,21	0,2977	0,35	0,022
32	0,0315	3,26	0,10	0,2959	0,35	0,011
64	0,0465	2,21	0,034	0,2964	0,35	0,005
128	0,0433	2,37	0,019	0,2976	0,35	0,003



Hình 1: *Thời gian chạy theo số luồng (thang log, thanh sai số là độ lệch chuẩn)*



Hình 2: *Speedup theo số luồng, so với đường lý tưởng $S = N$*



Hình 3: *Hiệu suất song song $E = S/N$*

4.2 Độ chính xác của π

Trong bảng thời gian, 5 lần chạy của một cấu hình dùng cùng seed 42 để cùng làm một khối việc, nên π của 5 lần giống hệt nhau và chỉ thời gian thay đổi. Để xem π dao động giữa các lần, `scripts/run_precision.sh` chạy thêm 5 lần với 5 seed khác nhau, cách nhau 1000. Seed phải cách xa nhau vì luồng `tid` dùng seed `seed + tid`: hai lần chạy lệch nhau 1 seed sẽ dùng lại 3 trên 4 dãy số ngẫu nhiên, và không còn độc lập.

Bảng 4: π *của 5 lần chạy với 5 seed khác nhau,* $n = 10^7$, $N = 4$ *(mutex dùng* $k = 1000$*)*

Lần	Seed	Một luồng	Cục bộ	Mutex
1	42	3,140532	3,141520	3,141520
2	1042	3,142388	3,141289	3,141289
3	2042	3,140790	3,141713	3,141713
4	3042	3,142003	3,141425	3,141425
5	4042	3,141239	3,141278	3,141278
Trung bình		3,141390	3,141445	3,141445
Độ lệch chuẩn		0,00079	0,00018	0,00018
$\lvert\bar{\pi} - \pi\rvert$		0,00020	0,00015	0,00015

Mọi giá trị nằm trong khoảng 3,1403–3,1424, khớp sai số chuẩn $5{,}2 \cdot 10^{-4}$ của công thức (2). Approach 3 cho đúng từng chữ số như Approach 2 ở cả 5 lần. `rand_r` của glibc cho cùng một dãy số trên mọi máy, nên các giá trị π ở đây không phụ thuộc máy đo.

Muốn đúng 5–6 chữ số thập phân. Theo (2), mỗi chữ số đúng thêm cần số điểm tăng 100 lần: 5 chữ số (sai số $\sim 3 \cdot 10^{-6}$) cần khoảng $3\text{–}5 \cdot 10^{11}$ điểm, 6 chữ số cần khoảng $4 \cdot 10^{13}$. `rand_r` không làm được việc này: trạng thái của nó chỉ có 32 bit nên dãy số lặp lại sau tối đa 2^{32} số, tức khoảng $2 \cdot 10^9$ điểm. Kiểm định χ^2 trên cặp (x, y) liên tiếp với 10^7 cặp cũng cho 8 588, thấp hơn hẳn khoảng mong đợi [9 724; 10 278]: điểm của bộ sinh đồng dư tuyến tính rải *đều hơn* ngẫu nhiên thật. Phân bố một chiều thì đạt (χ^2 = 925,5 với 999 bậc tự do), và hai luồng seed 42 và 43 có hệ số tương quan $-0{,}0003$.

Vì vậy nhóm viết thêm `src/pi_big.c`: cấu trúc giống Approach 2, nhưng mỗi luồng dùng bộ sinh 64 bit xoshiro256+ (chu kỳ $2^{256} - 1$, khởi tạo bằng splitmix64 để các luồng không trùng dãy), số thực 53 bit và biến đếm 64 bit.

Bảng 5: *pi_big, $N = 4$, seed 42 (results/precision.csv)*

n	$\hat{\pi}$	$\lvert\hat{\pi} - \pi\rvert$	$\sigma_{\hat{\pi}}$
10^7	3,1419652	$3{,}7 \cdot 10^{-4}$	$5{,}2 \cdot 10^{-4}$
10^8	3,1416643	$7{,}2 \cdot 10^{-5}$	$1{,}6 \cdot 10^{-4}$
10^9	3,1415584	$3{,}4 \cdot 10^{-5}$	$5{,}2 \cdot 10^{-5}$
10^{10}	3,1415802	$1{,}2 \cdot 10^{-5}$	$1{,}6 \cdot 10^{-5}$
10^{11}	**3,1415946**	$2{,}0 \cdot 10^{-6}$	$5{,}2 \cdot 10^{-6}$
$5 \cdot 10^{11}$	**3,1415960**	$3{,}3 \cdot 10^{-6}$	$2{,}3 \cdot 10^{-6}$

Với 10^{11} điểm trở lên, kết quả đúng 5 chữ số thập phân 3,14159. Lần chạy $5 \cdot 10^{11}$ điểm mất **[TODO: thời gian trên máy đo, cột time_sec của results/precision.csv]** giây trên máy đo 4 vCPU. Sáu chữ số cần khoảng $4 \cdot 10^{13}$ điểm, tức gấp 80 lần lần chạy lớn nhất ở trên, khoảng vài giờ: nhóm không chạy trong khuôn khổ bài. Đây là giới hạn cố hữu của Monte Carlo: sai số chỉ giảm theo $1/\sqrt{n}$.

5. Phân tích

5.1 A. Speedup không tuyến tính

Định luật Amdahl (slide chương 4, *Amdahl's Law*) giới hạn speedup của chương trình có phần tuần tự s:

$$S(N) \leq \frac{1}{s + (1-s)/N}. \tag{4}$$

Vòng lặp Monte Carlo gần như song song hoàn hảo; phần tuần tự chỉ là tạo luồng, join và cộng gộp. Đo được $S(2) = 1{,}87$ và $S(4) = 3{,}58$, thấp hơn lý tưởng 2 và 4. Giải $S(4) = 3{,}58$ theo Amdahl cho $s \approx 3{,}9\,\%$. Con số này lớn hơn phần tuần tự thật của chương trình, vì nó còn gánh những chi phí mà công thức Amdahl không mô hình hoá: các nhân của máy ảo do máy chủ định thời, nên có lúc một vCPU bị dừng; xung nhịp CPU có thể hạ khi nhiều nhân cùng bận; và join phải chờ luồng chậm nhất. Ngay ở $N = 1$, Approach 2 chậm hơn Approach 1 khoảng 4 % (0,1066 so với 0,1028 s), là chi phí tạo một luồng và join.

5.2 B. Điểm bão hoà

Các luồng chỉ thật sự chạy song song tới số nhân mà hệ điều hành định thời được, ở đây `nproc` = 4. Thời gian tốt nhất của Approach 2 là 0,0287 s tại $N = 4$, đúng bằng số vCPU. Từ 4 lên 8, 16, 32 luồng thời gian gần như không đổi (0,0287 $\rightarrow$ 0,0315 s): thêm luồng không thêm nhân, các luồng chỉ chia nhau 4 nhân sẵn có theo lát thời gian. Đường cong này giống đồ thị mẫu của đề (máy 4 nhân, đỉnh ở 4 luồng).

5.3 C. Quá tải luồng (oversubscription)

Khi số luồng lớn hơn số nhân, bộ định thời (chương 5) cho từng luồng sẵn sàng chạy một lát thời gian theo lượt. Tổng thời gian CPU không đổi nên lượng việc có ích mỗi giây không tăng, trong khi mỗi luồng thêm vào kéo theo chi phí:

- chuyển ngữ cảnh mỗi lần bộ định thời đổi luồng trên một nhân;
- cache và bộ dự đoán rẽ nhánh phải nạp lại sau mỗi lần đổi;
- thời gian tạo và join của từng luồng;
- luồng hoàn thành không đều: join chờ luồng cuối, nên một luồng bị tạm ngưng gần cuối làm chậm cả lần chạy.

Số đo cho thấy đúng điều đó: 32 luồng mất 0,0315 s, 64 luồng 0,0465 s (chậm hơn 62 % so với $N = 4$). Độ lệch chuẩn cũng tăng vọt, từ $\leq 0{,}002$ s khi $N \leq 32$ lên 0,012 s ở $N = 64$ và 0,010 s ở $N = 128$: thời gian chạy phụ thuộc vào thứ tự định thời, mỗi lần một khác. $N = 128$ nhanh hơn $N = 64$ một chút (0,0433 so với 0,0465 s), nằm trong độ lệch chuẩn của hai điểm này nên không kết luận được gì.

5.4 D. Chi phí đồng bộ

Lệnh tăng `g_inside` là vùng tranh chấp (critical section, chương 6): mỗi lúc chỉ một luồng được thực thi. Với $k = 1$, mọi vòng lặp đều cần khoá:

- **Tranh chấp.** Khi mutex rảnh, `pthread_mutex_lock` chỉ là một lệnh nguyên tử trong user space. Khi luồng khác đang giữ, luồng gọi phải ngủ trong kernel qua system call `futex`, và luồng nhả khoá phải gọi thêm một system call để đánh thức nó. Mỗi lần ngủ và thức là một lần chuyển ngữ cảnh, đắt hơn nhiều so với vài phép tính của một điểm Monte Carlo.
- **Tuần tự hoá.** Dù có bao nhiêu luồng, các lượt cộng vẫn diễn ra lần lượt từng cái, nên phần việc quan trọng của vòng lặp không nhanh lên được.
- **Chuyển cache line.** Mutex và biến đếm nằm ở vùng nhớ mà mọi nhân đều ghi. Mỗi lần lấy khoá, cache line đó phải chuyển sang nhân đang lấy, các nhân liên tục làm mất hiệu lực bản sao của nhau.

Ở $N = 1$ (không có tranh chấp), riêng chi phí khoá và mở khoá đã là $0{,}1285 - 0{,}1028 = 0{,}026\,\text{s}$ cho 10^7 lượt, khoảng 2,6 ns mỗi cặp. Ở $N = 4$, Approach 3 mất 0,2838 s so với 0,0287 s của Approach 2: **chậm 9,9 lần**, và chậm gần 3 lần so với chạy một luồng. Từ $N = 2$ trở đi thời gian gần như hằng số (0,27–0,30 s): thêm luồng chỉ thêm người xếp hàng trước cùng một khoá.

5.5 E. Hiệu suất song song

$E(N)$ cho biết bao nhiêu phần năng lực của mỗi luồng thêm vào biến thành speedup có ích. Với Approach 2, E giữ ở 0,90–0,96 khi $N \leq 4$ (mỗi luồng có nhân riêng), rồi giảm gần đúng theo $4/N$ khi $N > 4$ vì S không tăng nữa: 0,44 ở $N = 8$, 0,10 ở $N = 32$, 0,019 ở $N = 128$. Với Approach 3, khoá làm E rơi ngay từ luồng thứ hai: 0,19 ở $N = 2$ và 0,09 ở $N = 4$.

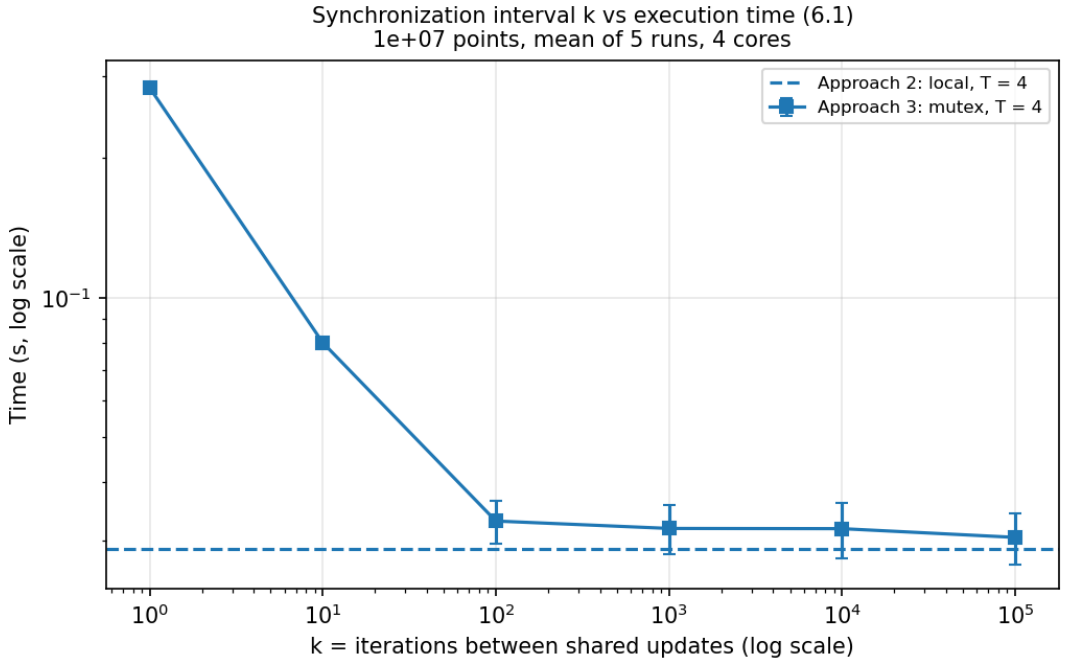
6. Thí nghiệm đánh đổi

6.1 Tần suất đồng bộ

Chạy Approach 3 với $N = 4$ và k = 1, 10, 100, 1 000, 10 000, 100 000. Mỗi luồng giờ chỉ lấy khoá khoảng $n/(Nk)$ lần thay vì mỗi điểm một lần, nên chi phí tranh chấp giảm theo k.

Bảng 6: *Approach 3 với $N = 4$ theo k; Approach 2 cùng N mất 0,0287 s*

k	Số lần lấy khoá mỗi luồng	T (s)	So với Approach 2
1	2 500 000	0,2838	×9,9
10	250 000	0,0800	×2,8
100	25 000	0,0330	+15 %
1 000	2 500	0,0318	+11 %
10 000	250	0,0318	+11 %
100 000	25	0,0304	+6 %



Hình 4: *Thời gian Approach 3 theo khoảng đồng bộ k ($N = 4$), đường ngang là Approach 2*

Từ $k = 100$, thời gian còn cách Approach 2 trong khoảng 6–15 %, chênh lệch cỡ độ lệch chuẩn của các điểm này (0,003–0,004 s). Phần còn lại là phép chia lấy dư `(i + 1) % k` ở mọi vòng lặp, Approach 2 không có. Kết luận: đồng bộ thưa đủ mức thì chi phí khoá gần như biến mất.

6.2 Mở rộng số luồng

Dải số luồng phủ ba vùng: ít hơn số nhân ($N < 4$), bằng số nhân ($N = 4$) và lớn hơn tới 128. Hình 1 và 2 cho đúng hình dạng dự kiến: thời gian giảm tới $N = 4$, đi ngang tới $N = 32$, rồi tăng ở $N = 64$–128 do quá tải luồng (mục C).

6.3 Tính cục bộ và tính dùng chung

Bảng 7: *So sánh Approach 2 và Approach 3 ($k = 1$)*

	Approach 2 (cục bộ)	Approach 3 (dùng chung, $k = 1$)
Vùng tranh chấp	không có trong vòng lặp, chỉ bước cộng gộp sau join	lệnh tăng biến đếm, mỗi vòng một lần
Tranh chấp khoá	không	mọi luồng tranh một mutex
Tuần tự hoá	không trong lúc tính	mọi lượt cộng xếp hàng lần lượt
T tại $N = 4$	0,0287 s	0,2838 s

7. Các thực hành tốt rút ra

Mỗi thực hành đi kèm một số đo của nhóm.

1. **Đếm cục bộ, gộp một lần (map–reduce).** Ở $N = 4$: 0,0287 s so với 0,2838 s khi dùng biến chung có khoá.

2. **Số luồng bằng số nhân.** Speedup dừng ở $N = 4$ = `nproc`; thêm luồng không nhanh hơn, quá nhiều luồng ($N = 64$) còn chậm hơn 62 %.

3. **Đồng bộ càng thưa càng tốt.** Với $k \geq 100$, bản dùng mutex chỉ chậm hơn bản không khoá 6–15 %.

4. **Mỗi luồng một trạng thái sinh số ngẫu nhiên.** Dùng `rand_r` với seed riêng; `rand()` có trạng thái chung sẽ xếp hàng các luồng sau một khoá ẩn.

5. **Dữ liệu riêng của luồng không nằm chung cache line.** Đếm trong biến stack, không trong `count[tid]` (false sharing).

6. **Đo thời gian đúng cách.** Dùng `CLOCK_MONOTONIC`, không dùng `clock()`; chạy nhiều lần, báo trung bình và độ lệch. Ở $N \geq 64$ độ lệch chuẩn tăng gấp 5 lần, một lần đo duy nhất sẽ cho kết luận sai.

7. **Kiểm tra đúng sai, không chỉ tốc độ.** Cùng seed thì `inside` phải giống nhau giữa các approach; bỏ khoá thì mất khoảng 78 % lượt cộng.

8. **Chọn bộ sinh số ngẫu nhiên đủ cho độ chính xác cần.** `rand_r` 32 bit lặp lại sau $\sim 2 \cdot 10^9$ điểm; muốn 5 chữ số thập phân thì cần bộ sinh 64 bit và $\geq 10^{11}$ điểm.

8. Kết luận

Ước lượng song song của bài toán gần như không có phần tuần tự đạt speedup 3,58 trên 4 vCPU, bão hoà đúng tại số nhân và chậm lại khi quá tải luồng. Khoá mutex ở mọi vòng lặp làm chương trình chậm hơn cả một luồng; đồng bộ thưa ($k \geq 100$) xoá gần hết chi phí đó. Về độ chính xác, 10^7 điểm cho khoảng 3 chữ số thập phân đúng; 5 chữ số cần 10^{11} điểm và một bộ sinh số 64 bit.

Sử dụng công cụ AI

Theo quy định của đề cương môn học: **[TODO: nhóm liệt kê công cụ AI đã dùng và dùng vào việc gì, ví dụ Claude Code dùng để lập kế hoạch chia việc, viết script đo/vẽ, review pull request và soạn báo cáo]**. Mọi số liệu trong báo cáo đến từ việc chạy mã của nhóm trên máy đo.